earth-power-n2 상세 기술문서

Source-Code-Driven Django Architecture & Implementation Analysis

1. 문서 목적 및 작성 원칙

본 문서는 withlionbuddha/earth-power-n2 저장소의 ground 브랜치에서 실제 확인되는 소스 코드, 설정 파일, 의존성 정의, 실행 스크립트를 근거로 작성한다. 문서 작성 방식은 withlionbuddha/technical-documents 의 Implementation-Based, Traceable, Reference-Based, Verification-Aware 원칙을 적용한다.

따라서 파일이나 의존성이 존재한다는 사실과 실제 기능이 실행 가능한 상태라는 사실을 구분한다. 소스 코드에서 구현 근거가 확인되면 Implemented, 구현 요소가 있으나 실행 확인이 필요한 경우 Runtime Verification Required, 분석 범위에서 구현을 찾지 못한 경우 Not Implemented 로 구분한다.

2. 분석 대상과 결론 요약

영역	정적 분석 결론	핵심 근거
Framework	Django 기반	manage.py, config/asgi.py, config/wsgi.py, Django 5.2 dependency
Architecture	config / gateway / domain / common 분리	src 구조
User Domain	Custom User Model 존재	User(AbstractUser)
Routing	계층형 라우팅 의도 존재, 일부 구성 검증 필요	config.urls → gateway.urls → domain.urls → User.urls
Settings	환경별 dev/prod 분리 의도 존재, 실제 env 모듈 누락	config/settings/__init__.py
API/View	실질 비즈니스 View 미구현	domain/views.py, User/views.py empty
Admin	User Model Admin 등록	admin.site.register(User)
Quality	pre-commit + flake8/pylint/mypy/pytest 구성	pre-commit-config, dev.txt
Production	Gunicorn/Sentry/WhiteNoise dependency 존재	prod.txt
Operations	runserver/collectstatic/migrate 등 Script 존재	scripts/*

3. Repository / Module Architecture

```
earth-power-n2/
├─ manage.py
├─ config/
│  ├─ __init__.py
│  ├─ settings/
│  │  └─ __init__.py
│  ├─ urls.py
│  ├─ asgi.py
│  └─ wsgi.py
├─ src/
│  ├─ common/
│  ├─ gateway/
│  │  └─ urls.py
```

```
│   └── domain/
│       ├── urls.py
│       ├── views.py
│       └── User/
│           ├── apps.py
│           ├── models.py
│           ├── admin.py
│           ├── urls.py
│           ├── views.py
│           └── tests.py
├── requirements/
│   ├── base.txt
│   ├── dev.txt
│   └── prod.txt
├── scripts/
└── docs/
```

구조적으로 config 는 Django 전역 설정과 Application Entry Point, gateway 는 외부 요청의 첫 라우팅 계층, domain 은 비즈니스 도메인 진입점, User 는 사용자 도메인 단위를 담당하도록 분리하려는 형태다. 다만 현재 common 은 실질 구현이 거의 없고, domain 및 User 의 View 구현도 비어 있어 아키텍처 골격 단계에 가깝다.

4. Application Bootstrap Architecture

4.1 manage.py.main()

manage.py 의 main()은 DJANGO_SETTINGS_MODULE 을 config.settings 로 설정한 뒤 django.core.management.execute_from_command_line(sys.argv)을 호출한다. 이는 migrate, makemigrations, runserver, collectstatic 등의 Django Management Command 가 모두 동일한 Settings Package 를 사용하도록 하는 진입점이다.

```
manage.py.main()
  ├── os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
  ├── import execute_from_command_line
  └── execute_from_command_line(sys.argv)
```

4.2 config.wsgi.application

config/wsgi.py 역시 DJANGO_SETTINGS_MODULE=config.settings 를 지정한 후 get_wsgi_application()으로 WSGI callable 을 생성한다. requirements/prod.txt 에 Gunicorn 이 포함되어 있으므로 정적 구조상 WSGI Production Serving 을 고려한 구성이 확인된다. 다만 Gunicorn 실행 명령과 실제 배포 설정은 별도 검증이 필요하다.

4.3 config.asgi.application

config/asgi.py 는 get_asgi_application()을 사용하여 ASGI callable 을 노출한다. 파일 주석에는 Uvicorn/Daphne 를 예시로 들고 있으나 requirements 에는 해당 ASGI Server dependency 가 확인되지 않는다. 따라서 ASGI Entry Point 는 Implemented 이지만 실제 ASGI Runtime 구성은 Not Identified 로 분류하는 것이 정확하다.

5. URL Routing Architecture

5.1 전체 요청 흐름

```
HTTP Request
   ↓
config.urls
   ↓
src.gateway.urls
   ↓
src.domain.urls
   ↓
src.domain.User.urls
```

```
    ↓
View / Django Admin
```

라우팅 구조는 Root URL 에서 Gateway 로 진입한 뒤 Domain, User 하위 URL 로 위임하는 계층적 구조를 의도한다.

5.2 config.urls

config/urls.py 는 path('', include('src.gateway.urls'))를 사용한다. 이 부분은 Django 의 표준 include() 패턴과 부합하며, Root Request 를 gateway URLConf 로 위임하는 구현 근거가 명확하다.

5.3 src.gateway.urls

src/gateway/urls.py 는 path('', domain.urls)를 사용한다. 일반적인 Django URLConf 연결은 include('src.domain.urls') 또는 include(domain.urls) 형태이므로, 현재 표현이 실제 urlpatterns Resolver 에서 정상 처리되는지는 Runtime 검증이 필요하다. 같은 파일에서 __debug__/ 경로에 debug_toolbar.urls 를 include 한다.

5.4 src.domain.urls

src/domain/urls.py 는 path('', urls.views)와 path('user/', User.urls)를 정의한다. 그러나 현재 파일 자체가 urls 모듈이며 . import urls 를 다시 수행하고, domain/views.py 는 비어 있다. 또한 path()의 두 번째 인자는 callable view 또는 include() 결과가 일반적인 사용 형태이므로 현재 구조는 수정 또는 실행 검증이 필요하다.

5.5 src.domain.User.urls

User URLConf 역시 path('', views), path('admin/', admin.site.urls)를 정의한다. User/views.py 가 비어 있으므로 루트 User 요청 처리 callable 은 확인되지 않는다. admin.site.urls 자체는 URLConf 역할을 하므로 보통 include 방식 또는 직접 path 연결이 가능하지만, 최종 Resolver 성공 여부는 Application Startup/Test 로 확인해야 한다.

5.6 Debug Toolbar 중복 등록

__debug__/ 경로가 gateway, domain, User URLConf 에 반복 추가되어 있다. 정적 코드상 Debug Toolbar 접근 경로를 여러 계층에서 중복 노출하는 구조이며, 유지보수 관점에서는 최상위 Development URLConf 한 곳에만 두는 편이 명확하다.

6. User Domain Architecture

6.1 User Model

src.domain.User.models.User 는 django.contrib.auth.models.AbstractUser 를 상속하고 추가 Field 나 Method 없이 pass 만 가진다. 따라서 Django 기본 Authentication User 의 동작을 유지하면서 향후 확장 가능한 Custom User Model 을 정의한 상태다.

```
AbstractUser
   ▲
   │ inheritance
User
   └─ no additional fields / methods
```

6.2 User Admin

src/domain/User/admin.py 는 admin.site.register(User)를 실행한다. 이는 Custom User Model 을 Django Admin Registry 에 등록하는 실제 구현이다. 다만 Settings 의 AUTH_USER_MODEL 값이 해당 Model 을 정확히 가리키는지는 config.settings.env.user 파일이 현재 브랜치에 없기 때문에 정적 검증할 수 없다.

6.3 User AppConfig

src/domain/User/apps.py 에는 MyAppConfig(AppConfig)가 존재하며 default_auto_field='django.db.models.BigAutoField', name='User'를 설정하고 ready()에서 초기화 메시지를 출력한다.

여기서 name='User'는 현재 패키지 실제 경로 src.domain.User 와 일치하지 않을 가능성이 있다. 또한 INSTALLED_APPS 에 이 AppConfig 가 등록되는 설정 파일 자체가 현재 브랜치에서 확인되지 않으므로 AppConfig 활성 여부는 Runtime Verification Required 다.

6.4 User 기능 구현 범위

기능	상태	근거
Custom User class	Implemented	models.User(AbstractUser)
Admin registration	Implemented	admin.site.register(User)
Custom User fields	Not Implemented	pass
Signup View/API	Not Implemented	User/views.py empty
Login View/API	Not Identified	auth endpoint 없음
Serializer	Not Implemented	serializer 파일 미확인
Permission/Role policy	Not Identified	별도 구현 미확인
User service layer	Not Implemented	service 모듈 미확인

7. Settings / Environment Architecture

7.1 load_dotenv()

config/settings/__init__.py 는 python-dotenv 의 load_dotenv()를 호출한다. 따라서 실행 시 .env 파일 또는 환경변수를 Python Process Environment 로 로드하려는 구현이 존재한다.

7.2 공통 설정 Import

```
from .env.common import *
from .env.rest import *
from .env.template import *
from .env.validator import *
from .env.user import *
```

설정 책임을 common/rest/template/validator/user 로 분리하려는 의도가 명확하다. REST Framework, Template, Validator, Custom User 와 관련된 설정을 기능 단위 Module 로 분리하려는 구조로 해석할 수 있다.

7.3 dev/prod 분기

```
ENV = os.getenv('DJANGO_ENV', 'dev')

if ENV == 'prod':
    import env.prod.*
else:
    import env.dev.*
```

환경별로 logging, database, cache, middleware, security, installed_app, resources 를 분리하려는 구조다. 이는 Development 와 Production 의 보안, DB, Cache, Logging 정책을 분리하기 위한 설정 아키텍처다.

7.4 현재 Branch 의 구조적 결함

현재 ground 브랜치에서 config/settings 디렉터리를 조회하면 __init__.py 만 존재한다. 즉 __init__.py 가 참조하는 env.common, env.rest, env.dev.*, env.prod.* 실제 파일이 저장소에서 확인되지 않는다. 이 상태 그대로라면 config.settings import 단계에서 ModuleNotFoundError 가능성이 높으며, 별도 비공개 배포 파일이 제공되는지 또는 누락된 것인지 확인이 필요하다.

7.5 설정값 콘솔 출력

settings/__init__.py 끝에서 DEBUG, ALLOWED_HOSTS, AUTH_USER_MODEL 을 print 한다. Development 확인에는 유용할 수 있으나 Production 에서는 설정정보 노출·로그 노이즈 관점에서 Logger 사용 또는 제거를 권장한다.

8. Dependency Architecture

8.1 Development

Dependency	역할	구현 판정
Django >=5.2.4,<6.0	Web Framework	Declared
djangorestframework >=3.15	REST API Framework	Declared, endpoint 미확인
django-cors-headers >=4.3	CORS Middleware	Declared, settings 미확인
django-debug-toolbar >=4.3	Development Debugging	Routing 확인
pytest / pytest-django	Testing	Declared, 실질 테스트 부족
pre-commit	Commit Hook 관리	Configured
black	Formatter	Declared
flake8	Static Analysis	Configured in pre-commit
pylint	Static Analysis	Configured in pre-commit
mypy	Type Checking	Configured in pre-commit
python-dotenv	Environment loading	실제 load_dotenv() 사용

8.2 Production

Dependency	역할	검증 상태
Django==5.2.4	Framework	Declared
DRF==3.15	REST API	Declared only
django-cors-headers==4.3	CORS	Declared only
gunicorn>=21.2	WSGI Server	Runtime Verification Required
sentry-sdk>=1.40	Error Tracking	Integration not identified
whitenoise>=6.6	Static Serving	Integration not identified
python-dotenv==1.0	Environment	load_dotenv() used

9. Code Quality / Test Architecture

9.1 pre-commit

.pre-commit-config.yaml 에는 flake8, pylint, mypy Hook 이 Commit Workflow 에 포함되고 pytest 는 stages:[manual]로 정의되어 있다. 따라서 정적 분석과 타입 검사를 자동화하고, Test 는 필요 시 수동 Hook 으로 실행하도록 구성하려는 정책이다.

9.2 Test Coverage

src/domain/User/tests.py 는 파일은 존재하지만 실질 테스트 코드가 확인되지 않는다. 따라서 Test Framework Dependency 와 pre-commit 설정은 존재하나 실제 Regression Test Suite 는 Not Implemented 로 분류해야 한다.

- 권장 Smoke Test: django.setup() 성공
- URL Resolver Test: reverse/resolve 또는 Client GET
- User Model Test: get_user_model()이 Custom User 를 반환하는지 확인
- Admin Registry Test: User 가 admin.site 에 등록되어 있는지 확인
- Settings Test: dev/prod import 및 필수 설정값 존재 여부 확인

10. Operations Script Analysis

Script	실제 내용	판정	주의사항
runserver.sh	python manage.py runserver 8080	Implemented	Development server 전용
collectstatic.sh	python manage.py collectstatic	Implemented	STATIC_ROOT 설정 필요
migrate.sh	DB 삭제 → makemigrations app → migrate	Artifact exists / Problematic	bash 에서 Windows del 사용, app 경로 불일치
superuser.sh	Superuser 생성 명령 존재	Implemented artifact	대화형 여부 및 환경 검증 필요
requirements.sh	Dependency install script	Implemented artifact	실제 target requirements 검증 필요

특히 migrate.sh 는 #!/bin/bash 를 선언하면서 del 명령을 사용한다. del 은 Windows CMD 명령이며 Bash 에서 일반적으로 사용할 수 없다. 또한 현재 Domain App 경로는 src/domain/User 인데 makemigrations app 을 호출하므로 App Label 도 현재 코드 구조와 불일치할 가능성이 높다. 따라서 이 파일은 'Migration 자동화 스크립트 존재'라는 구현 근거는 되지만 '정상 동작하는 Migration Pipeline'의 근거로 사용할 수 없다.

11. Request Processing Flow

```
[Client]
   |
   v
config.urls
   |
   v
src.gateway.urls
   |
   v
src.domain.urls
   |
   +--> /user/ ---> src.domain.User.urls
   |                  |
   |                  +--> /admin/ ---> Django Admin
   |                  |
   |                  +--> User View (현재 미구현)
   |
   +--> Domain View (현재 미구현)
```

현재 Request Routing 골격은 존재하지만 최종 Business View 가 비어 있어 일반 사용자 요청에 대한 유효 Response 생성 흐름은 완성되지 않았다.

12. Implementation Traceability Matrix

기술 항목	Source	Element	상태
Management Entry	manage.py	main()	Implemented
ASGI Entry	config/asgi.py	application	Implemented
WSGI Entry	config/wsgi.py	application	Implemented
Root Routing	config/urls.py	urlpatterns	Implemented
Gateway Routing	src/gateway/urls.py	urlpatterns	Runtime Verification Required
Domain Routing	src/domain/urls.py	urlpatterns	Runtime Verification Required
User Routing	src/domain/User/urls.py	urlpatterns	Runtime Verification Required
Custom User	src/domain/User/models.py	User	Implemented
Admin Registration	src/domain/User/admin.py	admin.site.register(User)	Implemented
App Init	src/domain/User/apps.py	MyAppConfig.ready()	Runtime Verification Required
Settings Loader	config/settings/__init__.py	load_dotenv / DJANGO_ENV	Implemented
Settings Modules	config/settings/env/*	referenced but absent	Missing
User API	src/domain/User/views.py	-	Not Implemented
Domain API	src/domain/views.py	-	Not Implemented
Static Analysis	.pre-commit-config.yaml	flake8/pylint/mypy	Configured
Tests	src/domain/User/tests.py	-	Not Implemented

13. Issues / Limitations

- config/settings/env 하위 모듈이 현재 브랜치에 없어 Settings Import 완결성이 없다.
- Gateway/Domain/User URLConf 일부가 Django 표준 include/callable 형태와 달라 Resolver 검증이 필요하다.
- domain/views.py 와 User/views.py 가 비어 있어 Business Endpoint 가 완성되지 않았다.
- User AppConfig.name='User'가 실제 Python package path 와 불일치할 가능성이 있다.
- Debug Toolbar URL 이 여러 URLConf 에 중복 등록되어 있다.
- requirements 에 DRF/CORS/Sentry/WhiteNoise 가 있으나 실제 적용 설정이 분석 범위에서 확인되지 않는다.
- migrate.sh 가 Bash 와 Windows CMD 명령을 혼합하고 App Label 도 현재 구조와 불일치할 가능성이 있다.
- Unit/Integration Test Suite 가 실질적으로 구현되지 않았다.

14. Improvement Recommendations

- config/settings/env 공통 및 dev/prod 설정 파일을 저장소에 포함하거나 별도 Secret/Deployment Artifact 로 관리한다면 그 제공 방식을 명확히 문서화한다.
- URLConf 를 include('src.domain.urls'), include('src.domain.User.urls') 방식으로 정리하고 URL Resolver Test 를 추가한다.
- User AppConfig.name 을 실제 import path 와 일치시키고 INSTALLED_APPS 등록을 테스트한다.
- User Serializer/ViewSet/API URL 을 추가하여 Custom User Model 의 실제 Application 기능을 구현한다.
- Debug Toolbar 는 DEBUG=True 인 Development Root URLConf 에서만 조건부 등록한다.
- DRF, CORS, Sentry, WhiteNoise 각각의 실제 settings 적용 여부를 코드로 연결한다.
- migrate.sh 를 OS 독립적 또는 Bash 호환 명령으로 수정하고 실제 App Label 기반으로 Migration 을 수행한다.
- pytest-django 기반 Startup, Routing, User Model, Admin, Settings 테스트를 최소 회귀 Test Set 으로 구축한다.

- Production 시작 명령(Gunicorn command), Environment Variables, Health Check, Static File 처리 정책을 별도 운영 문서로 명시한다.

15. Runtime Verification Plan

순서	검증 명령/방법	검증 목적	성공 기준
1	python manage.py check	Settings/App 기본 무결성	System check 오류 없음
2	python -c "import django; django.setup()"	Settings import	예외 없이 setup 완료
3	python manage.py showmigrations	App/Migration 인식	User 관련 app 인식
4	python manage.py makemigrations --check	Model/Migration drift	변경 필요 없음 또는 예상 변경
5	python manage.py migrate	DB schema 적용	모든 migration 성공
6	pytest	Test Suite	실패 0
7	python manage.py runserver 8080	Development boot	Server 정상 기동
8	URL Client Test	Routing	예상 status/response
9	Admin login	Custom User/Admin	User 관리 가능
10	gunicorn config.wsgi:application	Production WSGI	Worker 정상 기동

16. Architecture Summary

earth-power-n2 는 Django 기반 Web Application 의 초기 아키텍처 골격, Custom User Model, Admin 등록, 환경별 Settings 분리 의도, Development/Production Dependency 분리, 정적 분석 Hook 및 운영 Script 를 구현한 프로젝트다.

다만 현재 ground 브랜치 기준으로는 Settings env 모듈 누락, View/API 미구현, 일부 URLConf 구조 문제, Test 부재, Migration Script 불일치가 존재하므로 '완성된 운영 애플리케이션'보다는 'Django Backend Architecture 및 개발환경 구조를 구성하는 단계'로 기술하는 것이 소스 근거에 가장 충실하다.